

# **t-SNE**

t-distributed Stochastic Neighbor Embedding  
Visualisation de Données Haute Dimension

Cours de Machine Learning Non Supervisé

29 décembre 2025

## **Table des matières**

# 1 Introduction

Le **t-SNE** (t-distributed Stochastic Neighbor Embedding) est une technique de réduction de dimensionnalité développée par Laurens van der Maaten et Geoffrey Hinton en 2008.

## Concept Clé

### Objectif Principal

Contrairement à la PCA qui préserve les distances globales, t-SNE cherche à :

- Préserver les **relations de voisinage local**
- Projeter des données haute dimension en 2D ou 3D pour **visualisation**
- Révéler la **structure en clusters** cachée dans les données

t-SNE est principalement un outil de **visualisation**, pas de réduction de dimension pour le machine learning.

## 2 Pourquoi t-SNE après l'Espace Latent ?

### 2.1 Le Problème de Visualisation

Après avoir entraîné un autoencodeur, vous obtenez un espace latent de dimension  $d$  (par exemple,  $d = 2$  ou  $d = 32$ ).

## Exemple

### Cas Pratique : Votre Autoencodeur

- **Entrée** : 5 features (température, vitesse, couple, usure)
- **Espace latent** : 2 dimensions
- **Problème résolu ?** Oui, vous pouvez déjà visualiser en 2D !

### Mais si votre espace latent avait 32 dimensions ?

- Impossible de visualiser directement
- t-SNE permet de passer de 32D à 2D tout en préservant les clusters

### 2.2 Quand Utiliser t-SNE ?

| Situation            | t-SNE Utile ? | Raison                        |
|----------------------|---------------|-------------------------------|
| Espace latent 2D     | Non           | Déjà visualisable             |
| Espace latent 3D     | Optionnel     | Peut améliorer la séparation  |
| Espace latent $> 3D$ | Oui           | Indispensable pour visualiser |

## 3 Principe de Fonctionnement

### 3.1 Intuition

t-SNE fonctionne en deux étapes :

1. **Espace original** : Calculer les probabilités de voisinage entre tous les points
2. **Espace réduit** : Placer les points en 2D de façon à reproduire ces probabilités

## 3.2 Algorithme Simplifié

### Fondement Mathématique

#### Étape 1 : Similarités dans l'Espace Original

Pour chaque paire de points  $(x_i, x_j)$  en haute dimension, on calcule :

$$p_{j|i} = \frac{\exp(-||x_i - x_j||^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-||x_i - x_k||^2 / 2\sigma_i^2)}$$

Puis on symétrise :

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

**Interprétation :**  $p_{ij}$  mesure la probabilité que  $x_i$  choisisse  $x_j$  comme voisin.

### Fondement Mathématique

#### Étape 2 : Similarités dans l'Espace Réduit (2D)

Pour les points projetés  $(y_i, y_j)$  en 2D, on utilise une distribution de Student (t) :

$$q_{ij} = \frac{(1 + ||y_i - y_j||^2)^{-1}}{\sum_{k \neq l} (1 + ||y_k - y_l||^2)^{-1}}$$

#### Pourquoi la distribution t ?

- Elle a des queues plus lourdes qu'une gaussienne
- Permet de mieux séparer les clusters en 2D
- Évite le "crowding problem" (entassement des points)

### Fondement Mathématique

#### Étape 3 : Optimisation

On minimise la divergence de Kullback-Leibler entre  $P$  et  $Q$  :

$$\text{KL}(P||Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Cette optimisation se fait par descente de gradient.

## 4 Hyperparamètres Clés

### 4.1 Perplexité (Perplexity)

C'est le paramètre le plus important.

- **Définition :** Mesure du nombre effectif de voisins considérés pour chaque point
- **Valeur typique :** 5 à 50 (défaut : 30)
- **Effet :**
  - Perplexité faible (5-10) : Focus sur la structure locale (petits clusters)
  - Perplexité élevée (30-50) : Capture la structure globale

### Attention

#### Règle Empirique

La perplexité doit être inférieure au nombre de points :

$$5 \leq \text{perplexity} < \frac{n}{3}$$

Pour  $n = 10000$  points, tester : 30, 50, 100.

## 4.2 Nombre d'Itérations

- **Défaut** : 1000 itérations
- **Recommandation** : Au moins 1000, souvent 2000-5000 pour convergence
- **Indicateur** : Surveiller la valeur de KL divergence (doit décroître)

## 4.3 Learning Rate

- **Défaut** : 200
- **Plage** : 10 à 1000
- Si les points s'agglomèrent en boule : augmenter
- Si les points explosent : diminuer

# 5 Avantages et Limites

## 5.1 Avantages

- Excellente préservation de la structure locale
- Révèle des clusters non linéaires
- Visualisations très intuitives

## 5.2 Limites Critiques

### Attention

#### Limitations Importantes

1. **Non déterministe** : Deux exécutions donnent des résultats différents
2. **Distances non préservées** : Les distances en 2D n'ont PAS de sens absolu
3. **Lent** : Complexité  $O(n^2)$ , difficile au-delà de 10 000 points
4. **Pas de prédiction** : Ne peut pas projeter de nouvelles données

# 6 Exemple de Code Python

### Implémentation Python

```
1 from sklearn.manifold import TSNE
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # Supposons que Z est votre espace latent (n_samples, n_features)
6 # Par exemple, Z de dimension (10000, 32)
7
```

```

8 # 1. CONFIGURATION
9 tsne = TSNE(
10     n_components=2,          # Projection en 2D
11     perplexity=30,          # Nombre de voisins effectifs
12     n_iter=1000,            # Nombre d'iterations
13     random_state=42,        # Pour reproductibilit
14     verbose=1               # Affiche la progression
15 )
16
17 # 2. PROJECTION
18 print("Calcul du t-SNE en cours...")
19 Z_tsne = tsne.fit_transform(Z)
20
21 # 3. VISUALISATION
22 plt.figure(figsize=(10, 8))
23
24 # Colorer par les vraies tiquettes (si disponibles)
25 scatter = plt.scatter(Z_tsne[:, 0], Z_tsne[:, 1],
26                       c=y, cmap='viridis',
27                       alpha=0.6, s=10)
28 plt.colorbar(scatter, label='Machine Failure')
29 plt.title('Visualisation t-SNE de l'Espace Latent')
30 plt.xlabel('t-SNE Dimension 1')
31 plt.ylabel('t-SNE Dimension 2')
32 plt.grid(True, alpha=0.3)
33 plt.show()
34
35 # 4. ANALYSE PAR CLUSTER (si clustering fait)
36 # Exemple avec les clusters K-Means
37 plt.figure(figsize=(10, 8))
38 for cluster_id in np.unique(clusters):
39     mask = clusters == cluster_id
40     plt.scatter(Z_tsne[mask, 0], Z_tsne[mask, 1],
41               label=f'Cluster {cluster_id}', alpha=0.6, s=10)
42 plt.legend()
43 plt.title('t-SNE color par Clusters K-Means')
44 plt.show()

```

## 7 Cas d'Usage dans Votre Projet

### 7.1 Votre Situation Actuelle

- Espace latent : **2 dimensions** (déjà visualisable)
  - Clusters K-Means : 3 clusters identifiés
  - Clusters DBSCAN : 2 clusters identifiés
- Conclusion :** t-SNE n'est **pas nécessaire** pour votre cas actuel.

### 7.2 Quand Serait-il Utile ?

Si vous aviez un espace latent de dimension supérieure (ex : 32D), t-SNE permettrait de :

1. Visualiser la séparation entre Normal/Dégradé/Panne
2. Vérifier si les clusters trouvés par K-Means sont bien séparés visuellement
3. Identifier des sous-structures non détectées

## 8 Conclusion

t-SNE est un outil puissant de **visualisation exploratoire** pour comprendre la structure de données haute dimension. Cependant :

- Ne l'utilisez **pas** pour du machine learning (pas de prédiction)
- Ne l'utilisez **pas** si votre espace est déjà 2D ou 3D
- Utilisez-le pour **explorer** et **communiquer** vos résultats

Pour votre projet avec un espace latent 2D, vous pouvez directement visualiser  $Z$  sans t-SNE.